Troubleshooting Kubernetes Monitoring

> **Series:** K8S | **Notebook:** 9 of 13 | **Created:** January 2026 | **Last Updated:** 04/04/2026

Debugging Dynatrace Monitoring in Kubernetes

When monitoring doesn't work as expected, systematic troubleshooting is essential. This notebook covers common issues, diagnostic procedures, and resolution steps for Dynatrace Kubernetes monitoring.

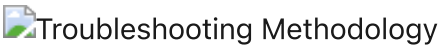
Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS/Managed access
kubectl	Configured for your cluster
Permissions	Cluster admin or debug access
Knowledge	K8S-01 through K8S-03

1. Troubleshooting Methodology

Systematic Approach



Common Symptom Categories

Symptom	Likely Component	Start Here
No cluster data	ActiveGate	Section 4
No node/pod data	OneAgent	Section 3
No app traces	Injection	Section 5
Partial data	Data collection	Section 6
Operator errors	Operator	Section 2

First Checks

```
# DynaKube status
kubectl -n dynatrace get dynakube

# All Dynatrace pods
kubectl -n dynatrace get pods

# Recent events
kubectl -n dynatrace get events --sort-by='.lastTimestamp' | tail -20
```

2. Operator Issues

Operator Not Running

Symptoms:
- DynaKube CR shows no status
- OneAgent pods not created

Diagnostic:

```
# Check operator deployment
kubectl -n dynatrace get deployment dynatrace-operator

# Check operator logs
kubectl -n dynatrace logs -l app.kubernetes.io/name=dynatrace-operator --
tail=100
```

Common Causes:

Cause	Resolution
Missing CRDs	Reinstall operator with Helm
RBAC issues	Check ServiceAccount permissions
Resource limits	Increase operator resources
Webhook failure	Check webhook certificates

DynaKube Stuck in Deploying

Diagnostic:

```
# Describe DynaKube
kubectl -n dynatrace describe dynakube dynakube

# Check conditions
kubectl -n dynatrace get dynakube -o jsonpath='{.items[*].status.conditions}'
| jq .
```

Common Causes:

Cause	Resolution
Invalid API token	Regenerate and update secret
Network policy blocking	Allow egress to Dynatrace
Image pull failure	Check pull secrets

3. OneAgent Issues

OneAgent Pods Not Starting

Diagnostic:

```
# Check DaemonSet
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

# Check pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=oneagent -o wide

# Describe failing pod
kubectl -n dynatrace describe pod &lt;oneagent-pod-name&gt;
```

Common Causes:

Cause	Resolution
Tolerations missing	Add tolerations to DynaKube

Cause	Resolution
Node selector mismatch	Update node selectors
Resource unavailable	Check node resources
Security context denied	Configure PSP/PSS

OneAgent Not Reporting

Diagnostic:

```
# Check OneAgent logs
kubectl -n dynatrace logs &lt;oneagent-pod&gt; -c oneagent --tail=100

# Check connectivity
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- curl -v
https://&lt;tenant&gt;.live.dynatrace.com/api/v1/deployment/installer/agent/c
onnectioninfo
```

Common Causes:

Cause	Resolution
Firewall blocking	Allow egress port 443
Proxy misconfiguration	Set proxy in DynaKube
Certificate issues	Check custom CA config

4. ActiveGate Issues

ActiveGate Not Starting

Diagnostic:

```
# Check StatefulSet
kubectl -n dynatrace get statefulset -l
app.kubernetes.io/component=activegate

# Check pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=activegate

# Check logs
kubectl -n dynatrace logs &lt;activegate-pod&gt; --tail=100
```

Common Causes:

Cause	Resolution
Memory too low	Increase resources in DynaKube
PVC issues	Check storage class
Token invalid	Regenerate API token

No Kubernetes Metrics

Diagnostic:

```
# Check capabilities
kubectl -n dynatrace get dynakube -o
jsonpath='{.items[*].spec.activeGate.capabilities}'

# Should include: kubernetes-monitoring
```

Resolution: Ensure `kubernetes-monitoring` capability is enabled:

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

5. Injection Issues

Pods Not Getting Injected

Diagnostic:

```
# Check namespace labels
kubectl get namespace &lt;namespace&gt; -o jsonpath='{.metadata.labels}'

# Check pod annotations
kubectl get pod &lt;pod&gt; -o jsonpath='{.metadata.annotations}'

# Check webhook
kubectl get mutatingwebhookconfigurations
```

Common Causes:

Cause	Resolution
Namespace not selected	Add matching labels
Pod has opt-out annotation	Remove annotation
Webhook not registered	Restart operator
CSI driver not mounted	Enable CSI in DynaKube

Check Injection Status

```
# Check if init container present
kubectl get pod &lt;pod&gt; -o jsonpath='{.spec.initContainers[*].name}'

# Check environment variables
kubectl exec &lt;pod&gt; -c &lt;container&gt; -- env | grep DT_
```

Namespace Selector Debugging

If using `namespaceSelector` in DynaKube:

```
# DynaKube selector
kubectl -n dynatrace get dynakube -o
jsonpath='{.items[*].spec.namespaceSelector}'

# Label a namespace for injection
kubectl label namespace &lt;namespace&gt; dynatrace-injection=enabled
```

6. Data Collection Issues

Missing Metrics

Diagnostic Steps:

1. Verify entity exists in Dynatrace
2. Check metric availability for entity type
3. Verify time range includes data
4. Check for filtering issues

```
// Verify Kubernetes entities are being discovered
fetch dt.entity.kubernetes_cluster
| fields entity.name, lifetime
| sort lifetime desc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_CLUSTER
// | fields name, lifetime
// | sort lifetime desc
```

```
// Check if container metrics are flowing
timeseries cpu = avg(dt.kubernetes.container.cpu_usage), from:-1h
| limit 1
```

```
// Check if logs are being ingested
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize logCount = count()
```

Missing Logs

Diagnostic:

```
# Check if log analytics is enabled
kubectl -n dynatrace get dynakube -o yaml | grep -A5 "env:"

# Verify OneAgent log collection
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- cat
/var/lib/dynatrace/oneagent/log/oneagent.log | tail -50
```

Common Causes:

Cause	Resolution
Log ingest disabled	Enable in tenant settings
Log volume too high	Configure sampling/filtering
Log format unsupported	Configure custom parsing

7. Connectivity Issues

Testing Connectivity

```
# From OneAgent pod
kubectl -n dynatrace exec &lt;oneagent-pod&gt; -c oneagent -- \
  curl -v https://&lt;tenant&gt;.live.dynatrace.com/api/v1/time

# From ActiveGate pod
kubectl -n dynatrace exec &lt;activegate-pod&gt; -- \
  curl -v https://&lt;tenant&gt;.live.dynatrace.com/api/v1/time
```

Network Policy Issues

```
# Allow Dynatrace egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dynatrace-egress
  namespace: dynatrace
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0
      ports:
        - protocol: TCP
          port: 443
```

Proxy Configuration

```
spec:
  proxy:
    value: https://proxy.example.com:8080
  # Or use a secret
  # proxy:
  #   valueFrom:
  #     secretKeyRef:
  #       name: proxy-secret
  #       key: proxy
```

8. Diagnostic Commands

Quick Health Check

```
#!/bin/bash
# dynatrace-health-check.sh

echo "=== DynaKube Status ==="
kubectl -n dynatrace get dynakube

echo "\n=== Dynatrace Pods ==="
kubectl -n dynatrace get pods

echo "\n=== OneAgent DaemonSet ==="
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

echo "\n=== ActiveGate StatefulSet ==="
kubectl -n dynatrace get statefulset -l
app.kubernetes.io/component=activegate

echo "\n=== Recent Events ==="
kubectl -n dynatrace get events --sort-by='.lastTimestamp' | tail -10
```

Collect Support Bundle

```
# Create support bundle
mkdir dynatrace-debug
cd dynatrace-debug

# DynaKube
kubectl -n dynatrace get dynakube -o yaml &gt; dynakube.yaml

# Pods
kubectl -n dynatrace get pods -o yaml &gt; pods.yaml

# Events
kubectl -n dynatrace get events &gt; events.txt

# Operator logs
kubectl -n dynatrace logs -l app.kubernetes.io/name=dynatrace-operator &gt;
operator.log

# OneAgent logs (one pod)
kubectl -n dynatrace logs $(kubectl -n dynatrace get pods -l
app.kubernetes.io/component=oneagent -o jsonpath='{.items[0].metadata.name}')
-c oneagent &gt; oneagent.log

# Package
cd ..
tar -czf dynatrace-debug.tar.gz dynatrace-debug/
```

Common kubectl Commands

Command	Purpose
`kubectl -n dynatrace get all`	All resources
`kubectl -n dynatrace describe dynakube`	DynaKube details
`kubectl -n dynatrace logs <pod>`	Pod logs
`kubectl -n dynatrace exec -it <pod> -- sh`	Shell access
`kubectl -n dynatrace port-forward <pod> 9999:9999`	Local port forward

9. DQL-Based Diagnostics

Complement kubectl diagnostics with DQL queries that detect Dynatrace component issues across your cluster fleet.

Common Failure Patterns

Pattern	Symptoms	Detection Method
CSI Volume Timeout	Pods stuck in ContainerCreating	Events with `MountVolume` + `timeout`
Injection Failure	Missing init container	Events with `Failed` + `dynatrace`
ActiveGate OOM	AG restarts, data gaps	Events with `OOMKilled` + `activegate`
OneAgent CrashLoop	Incomplete monitoring	Events with `CrashLoopBackOff` + `oneagent`
Connection Loss	Stale data, no updates	detected events with `AGENT_CONNECTION`

```
// Detect Dynatrace pod failures in the last 24h
fetch events, from:-24h
| filter event.kind == "K8S_EVENT"
| filter matchesPhrase(event.description, "dynatrace")
| filter matchesPhrase(event.description, "Failed") OR
        matchesPhrase(event.description, "Error") OR
        matchesPhrase(event.description, "BackOff") OR
        matchesPhrase(event.description, "OOMKilled") OR
        matchesPhrase(event.description, "CrashLoopBackOff")
| fields timestamp, event.description
| sort timestamp desc
| limit 50
```

```
// Detect ActiveGate connection issues via detected events
fetch dt.davis.events, from:-24h
| filter contains(toString(event.type), "ACTIVEGATE") OR
        contains(toString(event.type), "AGENT_CONNECTION")
| fields timestamp, event.type, event.category, affected_entity_ids
| sort timestamp desc
| limit 30
```

```
// Detect hosts with metric data gaps (missing CPU readings in last 6h)
timeseries from:-6h, interval:5m,
  cpuPoints = count(dt.host.cpu.usage),
  by:{dt.entity.host}
| fieldsAdd minPoints = arrayMin(cpuPoints)
| filter minPoints == 0
| fieldsAdd hostName = entityName(dt.entity.host, type:"dt.entity.host")
| fields hostName, minPoints
```

Summary

In this notebook, you learned:

- Systematic troubleshooting methodology
- Operator debugging and common issues
- OneAgent troubleshooting steps
- ActiveGate diagnostics
- Code injection debugging
- Data collection issue resolution
- Connectivity testing and network policy configuration
- Diagnostic commands for support bundle collection

References

- Dynatrace Operator Troubleshooting
- OneAgent Troubleshooting
- Kubernetes Monitoring FAQ
